

Temple University IR Project Information Retrieval System Report

Auto-export from project artifacts (`export_latex_reports.py`).

May 7, 2026

Contents

1 Abstract	1
2 Introduction	1
3 Data collection and normalization	1
4 Indexing	2
5 Experimental setup	2
6 Challenges and implementation notes	2
7 Results	3
7.1 Plots	3
7.2 Stopwords, queries, and optimization	7
8 Reproducibility	7

1 Abstract

We built an end-to-end IR pipeline over a Temple University web crawl: crawl, deduplicate, index, analyze the collection, construct annotated queries, pool candidates, and evaluate baseline retrieval models. This PDF summarizes the final artifacts and key results.

2 Introduction

Our goal was to build a reproducible retrieval system and quantify how standard models behave on a real web-derived collection. The dataset is a domain-specific crawl (primarily HTML pages) and the evaluation uses graded judgments where available.

3 Data collection and normalization

- Raw crawl JSONL lines: **71876** (`data/raw/corpus.jsonl`).
- Deduplicated documents: **42217** (`data/raw/corpus_deduped.jsonl`).

We deduplicated by keeping the first occurrence per docno and ensured stable identifiers for indexing.

4 Indexing

We indexed the deduplicated corpus into PyTerrier/Terrier (`index/terrier`) with `docno` meta-data and a tokenized text field. The resulting collection summary for this snapshot is: docs **42217.0**, vocab **214003.0**, tokens **37491119.0**, avg length **888.0573939408296**.

5 Experimental setup

We evaluated standard bag-of-words baselines (BM25, TF-IDF, and Query Likelihood) using graded qrels when present. We also ran a BM25 sensitivity sweep over (k_1, b) and a small optimization experiment comparing conjunctive vs disjunctive evaluation.

6 Challenges and implementation notes

System overview and challenges

End-to-end flow

1. **Crawl** (`src/crawl_temple.py`): BFS on `temple.edu`, stream-append JSONL, resume by scanning existing `docno` and URLs. Two queues prefer unseen URLs before re-fetching known pages (expansion-only).
2. **Dedupe** (`src/dedupe_corpus.py`):

	first	row	per	docno	→
data/raw/corpus_deduped.jsonl.					
3. **Index** (`src/build_index.py`): PyTerrier `IterDictIndexer`, meta `docno`, text field `text`.
4. **Analysis** (`src/collection_analysis.py`): vocabulary, Zipf, lengths, stopwords, word cloud → `outputs/analysis/`.
5. **Queries / pooling**: templates and `queries.tsv`; pool from BM25, TF-IDF, QL.
6. **Bootstrap qrels**: rank-1 BM25 labeled relevant (temporary metrics only).
7. **Eval, sensitivity, optimization**: PyTerrier experiments → `outputs/eval/`, `outputs/optimization/`.
8. **Reports**:

	<code>src/update_reports.py</code>	refreshes	Markdown;
<code>src/export_latex_reports.py</code> emits this PDF bundle.			

Snapshot numbers (from this export)

- Raw JSONL lines: **71876** (includes duplicate lines from restarts).
- Deduped documents: **42217**.
- Collection summary (first CSV row): docs 42217.0, vocab 214003.0, tokens 37491119.0, avg length 888.0573939408296.
- BM25 nDCG@10 (aggregate file, bootstrap qrels): **0.8300558702936255**.

Main challenges encountered

- **Resume and duplicates:** repeated crawl runs appended lines; some URLs mapped to multiple `docno` values over time. Dedupe-by-`docno` plus index-time `drop_duplicates` kept Terrier stable.
- **Frontier / scheduling:** treating “already saved” URLs as non-expandable starved BFS; fixed by always enqueueing unvisited links and then splitting *new* vs *expand* queues so new URLs are tried first.
- **Scale vs site reality:** many hosts return HTTP 403 to datacenter or script clients; tqdm only moves on *new* docs so long runs looked idle while HTTP work continued—postfix counters (`fetches`, queue sizes) were added for visibility.
- **PyTerrier/Java:** JVM init, absolute index paths, `text_attrs`, unique `docno` constraints required several integration passes.
- **Evaluation wiring:** PyTerrier Experiment metric tables differ in wide vs aggregate layouts; bootstrap `qrels` are not a substitute for human `qrels.tsv`.

7 Results

7.1 Plots

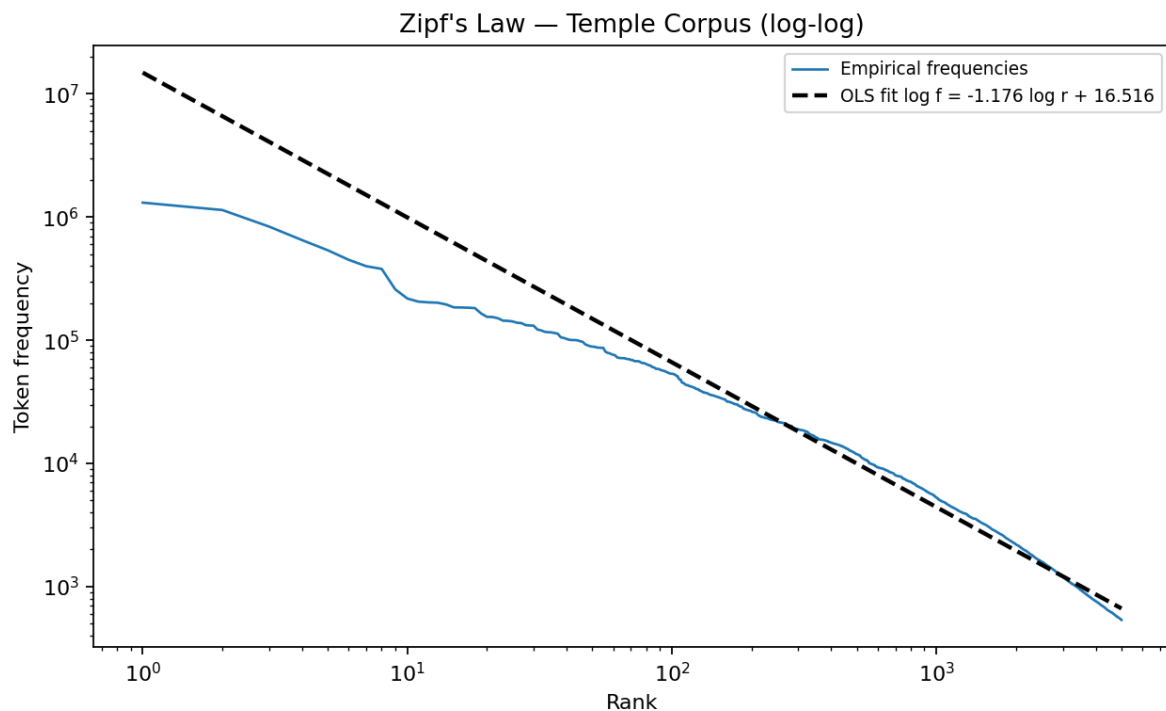


Figure 1: Zipf distribution (log–log) with dashed OLS fit.

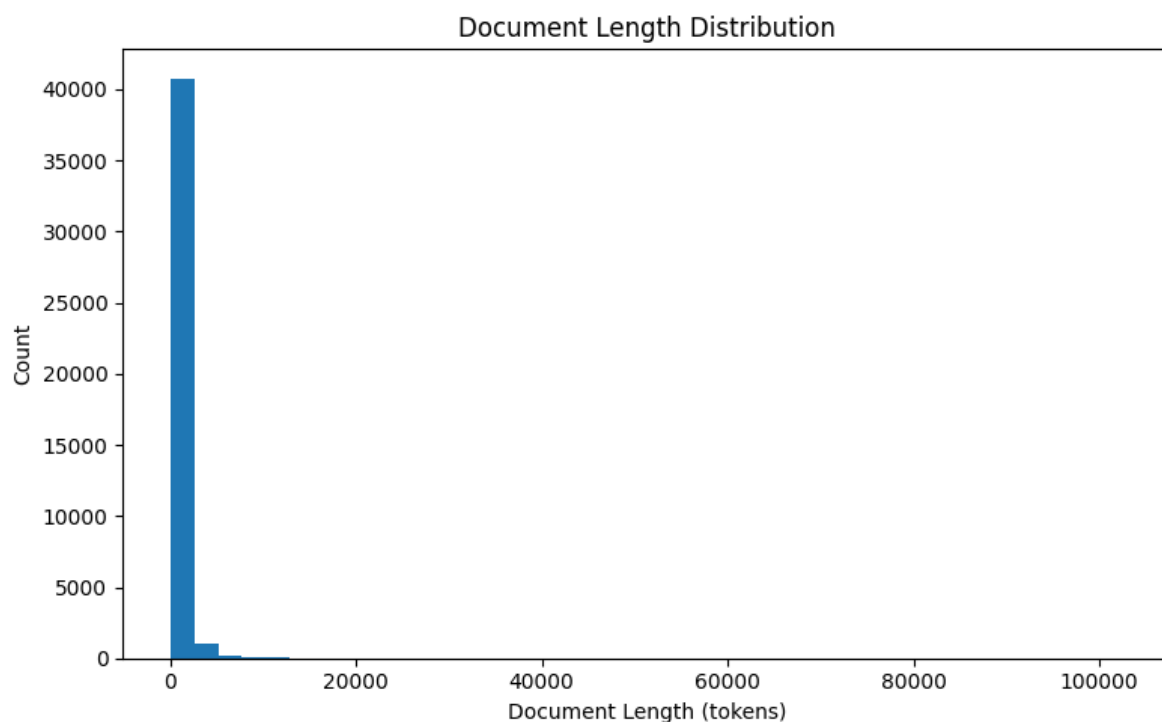


Figure 2: Histogram of tokenized document lengths.

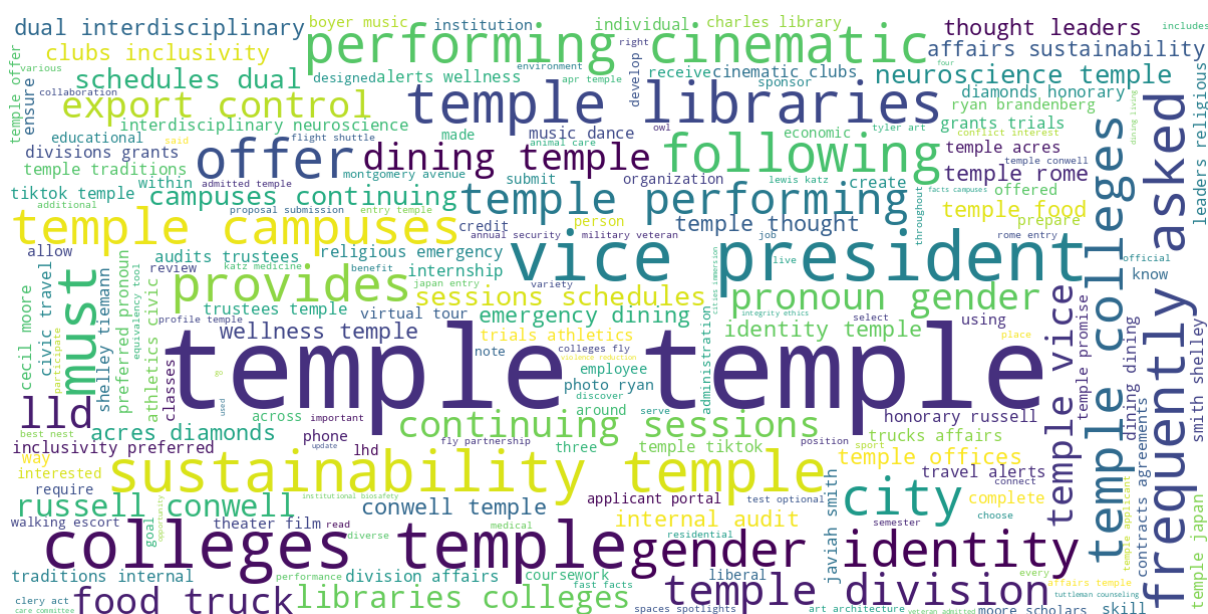


Figure 3: Word cloud of frequent non-stopwords.

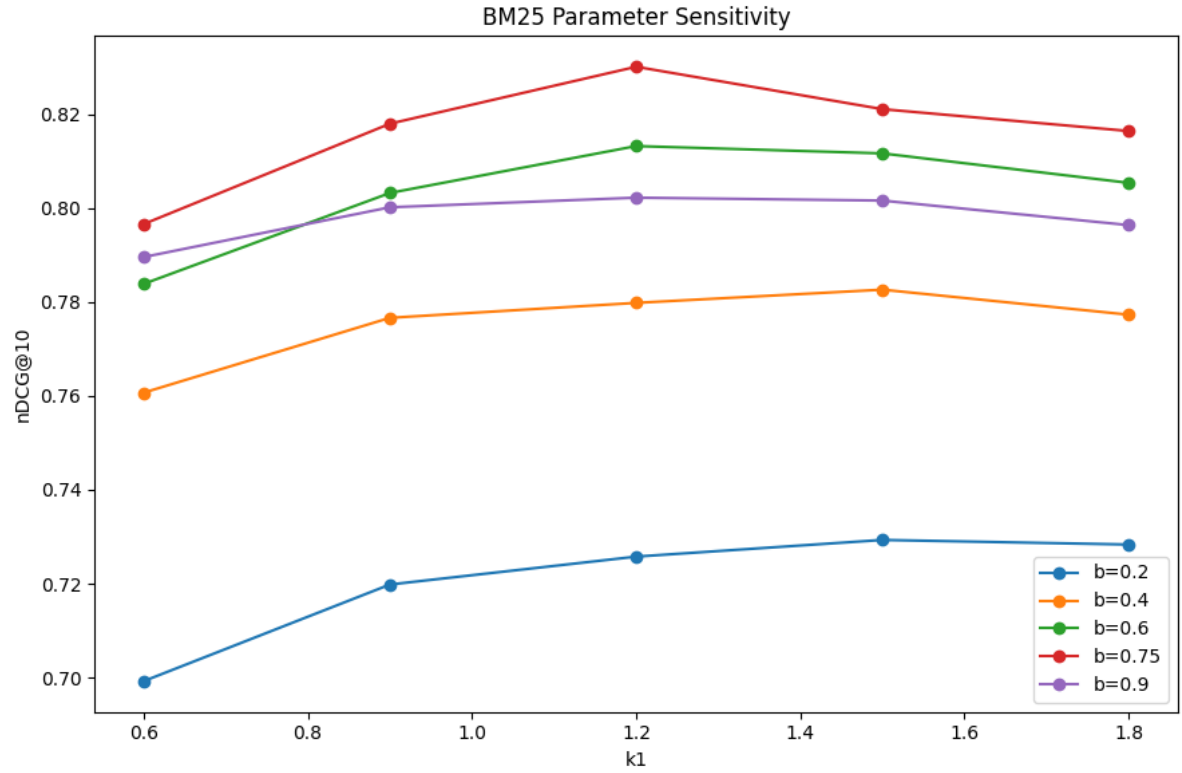


Figure 4: BM25 parameter sensitivity (nDCG@10 contours / curves).

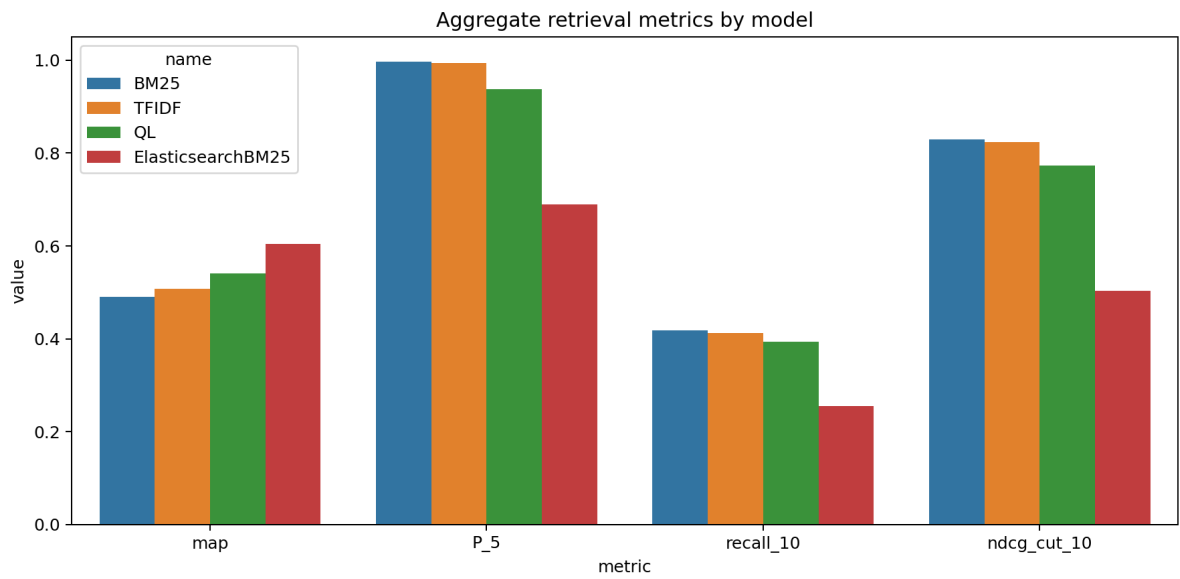


Figure 5: Aggregate retrieval metrics comparison.

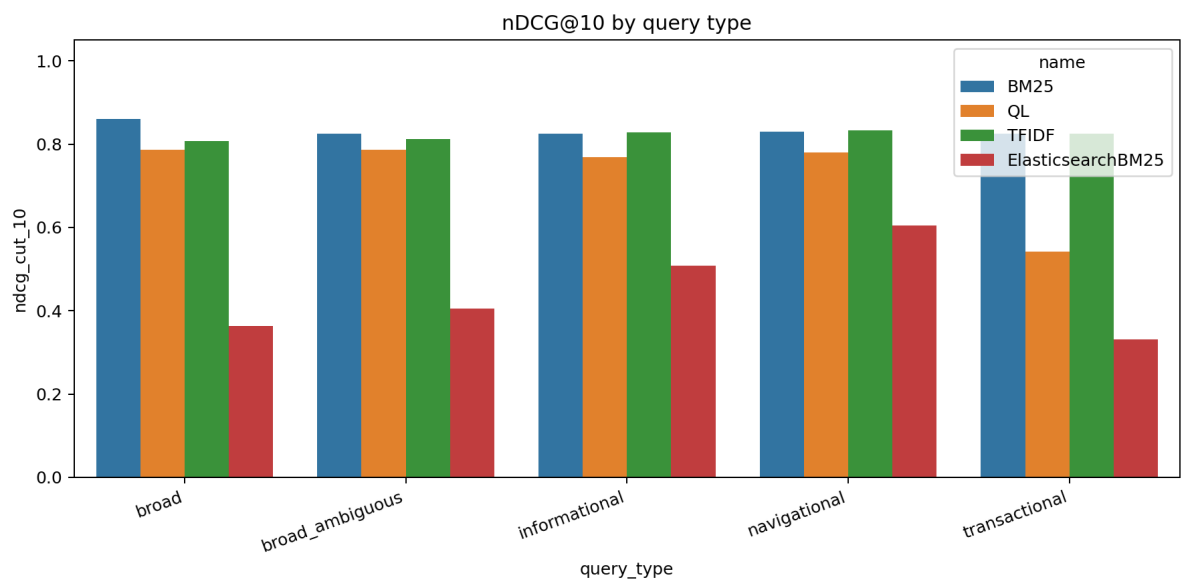


Figure 6: nDCG@10 broken down by query type.

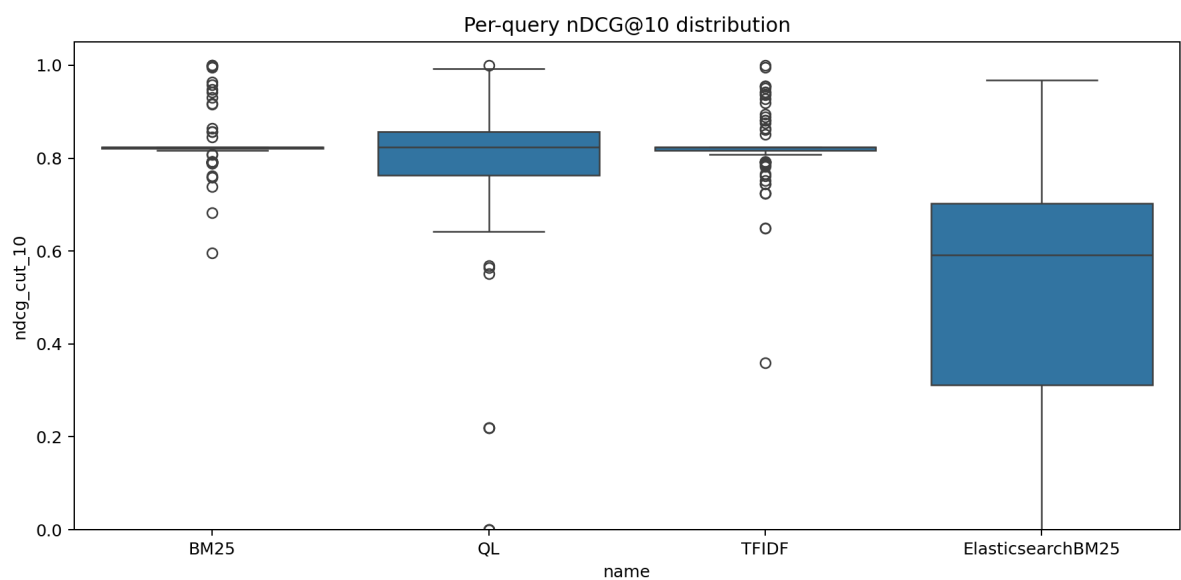


Figure 7: Dispersion of per-query nDCG@10.

7.2 Stopwords, queries, and optimization

Summary: stopwords

Token coverage uses NLTK English stopwords extended with corpus low-IDF terms (`corpus_tfidf_stopwords.csv` lists 250 corpus-specific candidates; the merged list has 448 types after union with NLTK). In a sample pass, 26.00% of tokens matched NLTK-only; 27.12% matched the TF-IDF low-IDF list only; after merging, 53.12% of tokens were treated as stopwords (vocabulary size from the summary file: 214003.0 types).

Summary: queries

The evaluation set contains 105 annotated queries in `data/processed/query_templates.tsv` (exported to `queries.tsv` for retrieval). Intent types: informational (39), navigational (34), broad_ambiguous (20), broad (10), transactional (2). Pooling and `qrels.tsv` judgments are described in the weekly sections; full tables remain under `outputs/eval/` if needed.

Summary: optimization

The optimization step compared **disjunctive** vs **conjunctive** BM25 query evaluation. Mean batch latency was about 1929 ms (disjunctive) vs 1618 ms (conjunctive); documents scored were 105000 vs 92390. Overlap at rank 10 between the two rankings was approximately 0.520.

Metrics and models

The figures above summarize sensitivity, query-type effects, and dispersion of per-query scores. The table below gives aggregate MAP, P@5, recall@10, and nDCG@10 for each run model. Use the exported per-query CSV for head-to-head contrasts beyond these totals.

Table 1: Aggregate retrieval metrics (graded qrels as in the eval run). PyTerrier baselines; optional Elasticsearch BM25 row when the extension was executed.

name	map	P_5	recall_10	ndcg_cut_10
BM25	0.4897853550849345	0.9961904761904762	0.4184918391876504	0.8300558702936255
TFIDF	0.5078014044068315	0.9942857142857143	0.4127164492343494	0.8243925857440955
QL	0.5411758282026168	0.937142857142857	0.3935576203971061	0.7732880292938707
ElasticsearchBM25	0.6048140709232691	0.6895238095238095	0.25539404090465273	0.5027055516290783

Sensitivity grids, per-query scores, and query-type splits are on disk under `outputs/eval/`; charts under `outputs/charts/`.

Formal hypothesis tests

PyTerrier paired comparisons: `outputs/eval/paired_t_tests.csv`. Elasticsearch vs BM25: `outputs/extension/es_vs_pyterrier_ttest.csv`.

8 Reproducibility

To regenerate outputs and this PDF:

```
.venv/bin/python3 -m src.refresh_phase_outputs
cd reports/tex && pdflatex main.tex && pdflatex main.tex
```